Studio Nx

Backend(FastAPI/Python) 26 — create_access_token() · _create_token()

2026-10-07

Backend(FastAPI/Python) 26 — `create_access_token()` · `_create_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인 — Access Token 발급 · PRD 01 FR-103 · FR-105
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	25 입력 스키마 (P1-04 끝) · 27 `create_refresh_token()`

1. 하는 일

회원 번호로 **30분짜리 Access Token(JWT)** 을 만든다. 05 `decode_access_token()` 이 읽는 토큰을 만드는 쪽이다.

`create_access_token(7)` → `"eyJhbGciOiJIUzI1NiIs….eyJzdWIiOiI3Iiwi….kQ3x…"`

토큰 안의 칸	값	뜻
`sub`	`"7"`	토큰 주인 (회원 번호, 글자)
`type`	`"access"`	종류 — 리프레시 토큰(27)과 구분
`iat`	`1791339232`	만든 시각 (1970년부터 센 초)
`exp`	`1791341032`	만료 시각 = `iat` + 1800초 (30분)

함수	역할
`create_access_token()`	바깥에서 부르는 이름. 수명 30분을 정함
`_create_token()`	실제로 만드는 곳. 27 `create_refresh_token()` 도 같은 함수를 씀 (05 의 `_decode_token` 과 짝)

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/login` (32)	home.html 로그인 모달	이메일 · 비밀번호가 맞으면 응답 `{"access_token": …}` 으로 줌
P1-05 `POST /auth/refresh` (33)	모든 페이지 (`api.js` 01)	리프레시 쿠키가 유효하면 새 Access Token 을 줌

받은 토큰은 브라우저 메모리에 두고(`setAccessToken`), 모든 API 요청의 `Authorization: Bearer …` 에 붙는다 (JS 02). 30분이 지나면 서버(05)가 401 을 주고, 브라우저가 33 으로 새 토큰을 받는다.

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다. **`from datetime import UTC, datetime, timedelta` 를 이번에 추가했다.** 이 함수들은 그것과 `jwt` · `get_settings` 를 쓴다.

import	쓰는 곳
`UTC` · `datetime` · `timedelta`	지금 시각(UTC), 30분이라는 기간
`jwt`	`jwt.encode` — 토큰 만들기 (05 의 `jwt.decode` 와 반대)
`get_settings`	`.env` 의 `JWT_SECRET` · `jwt_access_minutes`(30)

함수

```
JWT_ALGORITHM = "HS256"


def create_access_token(user_id: int) -> str:
    minutes = get_settings().jwt_access_minutes
    return _create_token(user_id, "access", timedelta(minutes=minutes))


def _create_token(user_id: int, token_type: str, lifetime: timedelta) -> str:
    now = datetime.now(UTC)
    payload = {
        "sub": str(user_id),
        "type": token_type,
        "iat": now,
        "exp": now + lifetime,
    }
    return jwt.encode(payload, get_settings().jwt_secret, algorithm=JWT_ALGORITHM)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime, timedelta`	파이썬 기본 `datetime` 모듈에서 세 이름을 가져옴

식	뜻
`timedelta(minutes=30)`	30분이라는 **기간**
`datetime.now(UTC)`	지금 시각 (UTC, 시간대 표시 있음)
`now + lifetime`	30분 뒤 시각
`jwt.encode(payload, 비밀값, algorithm="HS256")`	머리 · 내용 · 서명 세 부분을 만들어 `.` 으로 이은 글자. `datetime` 값은 PyJWT 가 초 단위 숫자로 바꿔 넣음
`str(user_id)`	`7` → `"7"`. JWT 규격에서 `sub` 는 글자
`_create_token` 의 `_`	이 파일 안에서만 쓰는 함수 (05 의 `_decode_token` 과 같은 관례)

4. 흐름

1. 설정에서 수명(30분)을 읽는다.

2. 지금 시각(UTC)을 잰다.
3. `sub` · `type` · `iat` · `exp` 를 담는다.
4. 서버 비밀값으로 서명해 토큰 글자를 돌려준다.

5. 왜 이렇게

선택	이유
30분	토큰은 가진 사람을 인정하는 방식(Bearer)이라, 훔쳐 가도 오래 쓰지 못하게 짧게 둔다. 대신 리프레시(27 · 33)로 사용자는 계속 로그인된 것처럼 쓴다
수명은 `.env` 설정에서	코드를 고치지 않고 바꿀 수 있다 (`jwt_access_minutes`)
`type` 을 넣음	05 가 `type` 을 검사해, 3일짜리 리프레시 토큰을 Access 자리에 쓰지 못하게 막는다
`iat`(만든 시각) 추가 — 템플릿에 없음	같은 회원이 같은 초에 두 번 받지 않는 한 토큰이 매번 달라진다. 나중에 "이 시각 이전 토큰은 무효" 같은 처리를 할 수 있다. 05 는 이 칸을 요구하지 않으므로 영향 없음
만들기를 `_create_token` 으로 분리	27 `create_refresh_token` 은 수명 · 종류만 다르다. 한 줄로 끝난다
알고리즘은 05 와 같은 `JWT_ALGORITHM`	만드는 쪽과 읽는 쪽이 한 상수를 쓴다

토큰 내용은 암호화되지 않는다 (05 §2). 누구나 읽을 수 있으므로 회원 번호 외에 이메일 · 권한 같은 것은 넣지 않는다.

6. 시험 (`tests/test_security.py` 에 5개 추가 · 모두 14개)

시험	확인한 것
만들고 읽기	`decode_access_token(create_access_token(7)) == 7` — 05 와 짝이 맞음
내용	`sub` = `"7"`, `type` = `"access"`, `exp - iat` = 1800초
머리	`alg` = `HS256`
시각	`iat` 가 지금과 2초 안, `exp` = `iat` + 1800
회원마다 다름	7번 · 8번 토큰이 다름

05 의 시험에서 쓰던 `jwt.encode` 직접 만들기(`make_token`)는 그대로 둔다. 만료 · 위조 같은 이상한 토큰은 이 함수로 만들 수 없기 때문이다.

전체 시험 191개 통과, `ruff` 검사 통과.